Software Requirements Specification (SRS)

Municipal Helpdesk System

Field	Value
Project Name	Municipal Helpdesk System
Document Type	Software Requirements Specification
Version	1.0
Date	2026-02-23
Status	Draft
Classification	Full-Stack Web Application (Web Portal) — Functional Prototype

---

Table of Contents

1. Introduction
2. Overall Description
3. System Architecture & Tech Stack
4. Specific Requirements
5. Data Requirements
6. External Interface Requirements
7. Non-Functional Requirements
8. Appendices

---

1. Introduction

1.1 Purpose

This document provides a complete Software Requirements Specification for the **Municipal Helpdesk System**, an AI-assisted civic issue reporting and resolution platform. It describes all functional and non-functional requirements, system architecture, data models, workflows, user roles, and interface specifications needed to build a functional prototype.

This document is intended for: - **Developers** implementing the frontend and backend. - **Architects** designing the system and infrastructure. - **QA Engineers** deriving test cases. - **Project Stakeholders** reviewing scope and functionality.

1.2 Scope

The Municipal Helpdesk System enables citizens to report civic issues (e.g., potholes, damaged streetlights, sanitation problems) via a web portal. An AI module classifies the uploaded image and recommends a relevant municipal department. An administrator reviews, approves, or overrides the AI's recommendation and assigns the ticket to a specific department worker. The worker then resolves the issue and uploads proof, or flags false reports for admin review.

Key capabilities include: - Citizen self-service ticket creation with image upload and GPS geolocation capture. - AI-powered image classification for automatic department routing. - Admin dashboard for ticket review, assignment, and escalation management. - Worker interface for task execution, resolution proof upload, and false-report flagging. - Officer analytics dashboard with KPIs, charts, and SLA tracking. - Role-Based Access Control (RBAC) across all four user roles. - Email notifications on ticket status changes. - AES-256 encryption for sensitive data fields.

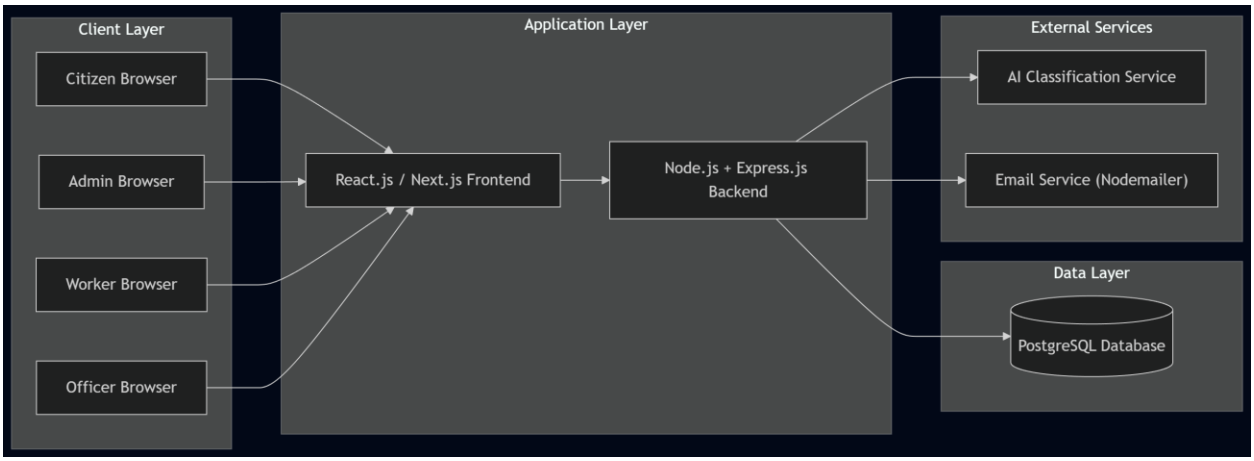
1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification
RBAC	Role-Based Access Control
JWT	JSON Web Token — used for stateless session management
ORM	Object-Relational Mapping
SLA	Service Level Agreement
KPI	Key Performance Indicator
OTP	One-Time Password
AES	Advanced Encryption Standard
CRUD	Create, Read, Update, Delete
API	Application Programming Interface
GPS	Global Positioning System
Ticket	A digital representation of a civic issue reported by a Citizen

2. Overall Description

2.1 Product Perspective

The Municipal Helpdesk System is a **self-contained, full-stack web application**. It is not a component of a larger existing system but is designed as an independent platform that integrates with external services (AI image classification and email delivery) via APIs.



2.2 Product Functions (High-Level)

#	Function	Description
1	User Authentication	Email/Password registration & login with JWT sessions and optional Email OTP.
2	Citizen Ticket Submission	Image upload, GPS capture, auto-population of contact info.
3	AI Image Classification	Automatic categorization of issue images into municipal departments.
4	Admin Ticket Management	Review AI recommendations, approve/override, assign to department workers.
5	Worker Task Execution	View assigned tickets, resolve with proof, or flag false reports.
6	Escalation Management	Admin handles worker-flagged tickets (reject or re-assign).
7	Master Data Management	Admin CRUD for Departments and Employee (Worker) accounts.
8	Officer Analytics Dashboard	Aggregated KPIs, charts, SLA tracking — read-only executive view.
9	Notification Service	Email notifications on all ticket status transitions.
10	Data Encryption	AES-256 encryption of highly sensitive user data fields.

2.3 User Roles & Permissions (RBAC)

The system enforces four strictly separated roles. Every API route and frontend page is protected by `requireRole()` middleware.

Role	Permissions
Citizen	Register/Login. Create tickets. View & track **own** tickets only. Cannot access

Role	Permissions
	any admin/worker/officer views.
Admin	Manage Departments & Users (CRUD). View all tickets. Review AI recommendations. Approve/Override/Assign tickets. Handle escalations.
Dept Worker	View tickets assigned to **self / own department** only. Mark tickets as Resolved (with proof photo). Flag false reports.
Officer	Read-only access to analytics dashboard. View aggregated KPIs, charts, SLA metrics. **Cannot** edit any tickets.

2.4 Operating Environment

Component	Specification
Client	Modern web browser (Chrome, Firefox, Edge, Safari)
Server OS	Linux / Windows / macOS (Node.js runtime)
Runtime	Node.js v18+
Database	PostgreSQL 14+
AI Service	Google Cloud Vision API or TensorFlow.js model
Email Service	SMTP-compatible server via Nodemailer

2.5 Design & Implementation Constraints

1. The system is a **functional prototype**; production hardening (load balancing, CDN, etc.) is out of scope.
2. Email sending may be **mocked** for the prototype, but the full logic path must exist.
3. AI image classification must support at least the categories: **Water, Electricity, Sanitation, Roads**.
4. All sensitive fields must use **AES-256 encryption** via Node.js `crypto` module.
5. The backend must follow a **modular, well-commented** codebase structure.

2.6 Assumptions & Dependencies

- Users have internet access and a GPS-capable device/browser.
- The AI classification model or API key is provisioned prior to deployment.
- A PostgreSQL database server is available and accessible.
- An SMTP server or mail service credentials are available for Nodemailer.

---

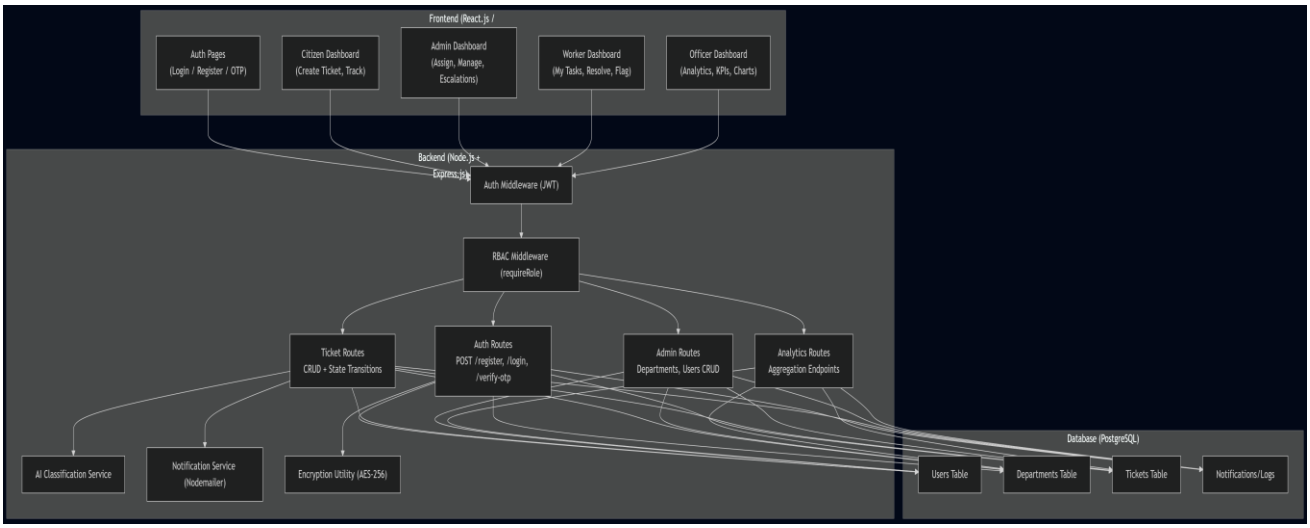
3. System Architecture & Tech Stack

3.1 Technology Stack

Layer	Technology
Frontend	React.js (or Next.js) + Tailwind CSS

Layer	Technology
Backend	Node.js + Express.js
Database	PostgreSQL (Relational)
ORM	Prisma or Sequelize
Auth	JWT for session management, bcrypt for password hashing
Encryption	Node.js `crypto` module — AES-256 for sensitive fields
AI	Google Cloud Vision API or pre-trained TensorFlow.js model
Email	Nodemailer (SMTP)

3.2 High-Level Architecture



3.3 Backend Module Structure (Recommended)

```
server/
├── config/
│   ├── db.js              # Database connection & ORM config
│   └── env.js             # Environment variables
├── middleware/
│   ├── auth.js            # JWT verification middleware
│   └── rbac.js            # Role-based access control middleware
├── models/
│   ├── User.js
│   ├── Department.js
│   ├── Ticket.js
│   └── Notification.js
├── routes/
│   ├── auth.routes.js
│   ├── ticket.routes.js
│   ├── admin.routes.js
│   └── analytics.routes.js
├── services/
```

```
│   ├── ai.service.js      # AI image classification
│   ├── email.service.js   # Nodemailer notification service
│   └── crypto.service.js  # AES-256 encryption utility
├── utils/
│   └── helpers.js
├── app.js                 # Express app setup
└── server.js              # Entry point
```

---

4. Specific Requirements

4.1 Functional Requirements

FR-01: User Registration

Field	Detail
ID	FR-01
Priority	High
Actor	Citizen (self-registration), Admin (creates Worker/Officer accounts)
Endpoint	`POST /api/auth/register`
Description	New Citizens can self-register with email and password. Admin can create Worker and Officer accounts and map Workers to Departments.

Acceptance Criteria: 1. System accepts `name`, `email`, `password`, `phone`, `address` fields. 2. Password is hashed using **bcrypt** before storage. 3. Sensitive fields (e.g., phone, address) are encrypted using **AES-256** before storage. 4. Duplicate email registration returns `409 Conflict`. 5. Default role for self-registration is `CITIZEN`. 6. Admin-created accounts can have role `DEPT_WORKER` or `OFFICER`. 7. Worker accounts require a `department_id` mapping.

FR-02: User Login

Field	Detail
ID	FR-02
Priority	High
Actor	All Roles
Endpoint	`POST /api/auth/login`
Description	Authenticate users via email/password and return a JWT token.

Acceptance Criteria: 1. System validates email and bcrypt-compared password. 2. On success, returns a signed JWT containing `userId`, `role`, `email`. 3. JWT has a configurable expiration (e.g., 24 hours). 4. Invalid credentials return `401 Unauthorized`.

FR-03: Email OTP Verification (Optional)

Field	Detail
ID	FR-03
Priority	Medium
Actor	Citizen
Endpoints	`POST /api/auth/send-otp`, `POST /api/auth/verify-otp`
Description	Optional email verification flow. Logic must fully exist; actual email send may be mocked.

Acceptance Criteria: 1. System generates a 6-digit OTP and associates it with the user's email. 2. OTP is valid for a configurable duration (e.g., 10 minutes). 3. `verify-otp` endpoint validates the OTP and marks the user as `email_verified = true`. 4. Expired or invalid OTP returns `400 Bad Request`.

FR-04: Citizen Ticket Creation (Workflow A)

Field	Detail
ID	FR-04
Priority	High
Actor	Citizen
Endpoint	`POST /api/tickets`
Description	Citizen creates a new ticket by uploading an image. System captures GPS, attaches contact info.

Acceptance Criteria: 1. Request accepts an **image file** (JPEG/PNG, max 10MB) via multipart form-data. 2. System captures **GPS coordinates** (latitude, longitude) from the client. 3. System auto-attaches the **Citizen's contact info** (name, email, phone) from their profile. 4. Citizen may optionally provide a text **description**. 5. Ticket is created with initial status: **`PENDING_AI`**. 6. System immediately triggers the AI classification pipeline (see FR-05). 7. Response returns the created ticket with an assigned `ticket_id`. 8. Requires `CITIZEN` role.

FR-05: AI Image Classification & Department Recommendation (Workflow B — Part 1)

Field	Detail
ID	FR-05
Priority	High
Actor	System (Automated)

Field	Detail
Trigger	Ticket created with status `PENDING_AI`
Description	AI service analyzes the uploaded image and sets a `recommended_department_id` on the ticket.

Acceptance Criteria: 1. AI service receives the uploaded image. 2. Classifies the image into one of the predefined municipal categories: **Water, Electricity, Sanitation, Roads** (extensible). 3. Maps the classification label to the corresponding `department_id` in the database. 4. Updates the ticket with `recommended_department_id` and `ai_confidence_score`. 5. Ticket status transitions to **`PENDING_ADMIN`**. 6. If AI classification fails, ticket still moves to `PENDING_ADMIN` with `recommended_department_id = null` and a flag indicating manual review is needed.

FR-06: Admin Ticket Review & Assignment (Workflow B — Part 2)

Field	Detail
ID	FR-06
Priority	High
Actor	Admin
Endpoint	`PUT /api/tickets/:id/assign`
Description	Admin reviews AI recommendation and assigns the ticket to a specific Worker in the chosen Department.

Acceptance Criteria: 1. Admin sees all tickets with status `PENDING_ADMIN` on their dashboard. 2. Each ticket displays the AI's `recommended_department_id` and `ai_confidence_score`. 3. Admin can **accept** the AI recommendation or **override** by selecting a different department. 4. Admin selects a specific **Worker** from the chosen department's roster. 5. System updates `assigned_department_id`, `assigned_worker_id`. 6. A boolean flag `ai_recommendation_accepted` records whether the admin accepted or overrode the AI. 7. Ticket status transitions to **`ASSIGNED`**. 8. **Email notification** sent to the Citizen informing them of the assignment. 9. Requires `ADMIN` role.

FR-07: Worker Task Resolution — Happy Path (Workflow C)

Field	Detail
ID	FR-07
Priority	High
Actor	Dept Worker
Endpoint	`PUT /api/tickets/:id/resolve`
Description	Worker completes the task, uploads an "After" photo as proof, and resolves

Field	Detail
	the ticket.

Acceptance Criteria: 1. Worker sees only tickets assigned to **themselves** or **their department**. 2. Worker uploads a **resolution image** ("After" photo) as proof of completion. 3. Worker may add **resolution notes**. 4. Ticket status transitions to **`RESOLVED`**. 5. **Email notification** sent to the Citizen confirming resolution. 6. Requires `DEPT_WORKER` role. 7. Worker can only resolve tickets with status `ASSIGNED`.

FR-08: Worker False Report Flagging (Workflow C — Edge Case)

Field	Detail
ID	FR-08
Priority	High
Actor	Dept Worker
Endpoint	`PUT /api/tickets/:id/flag-false`
Description	Worker flags a ticket as invalid/false report with written reasoning.

Acceptance Criteria: 1. Worker selects "Flag as False" on an assigned ticket. 2. Worker provides mandatory **written reasoning/proof** for the false-report claim. 3. Ticket status transitions to **`ESCALATED_TO_ADMIN`**. 4. Requires `DEPT_WORKER` role. 5. Only tickets with status `ASSIGNED` can be flagged.

FR-09: Admin Escalation Resolution

Field	Detail
ID	FR-09
Priority	High
Actor	Admin
Endpoint	`PUT /api/tickets/:id/resolve-escalation`
Description	Admin reviews escalated tickets and either rejects them or re-assigns them.

Acceptance Criteria: 1. Admin sees all tickets with status `ESCALATED_TO_ADMIN`. 2. Admin reviews the worker's reasoning for the false-report flag. 3. **Option A — Reject**: Admin marks the ticket as **`REJECTED`** (closed). Citizen notified. 4. **Option B — Re-assign**: Admin re-assigns the ticket to a different worker/department. Status returns to **`ASSIGNED`**. Worker notified. 5. Admin provides **resolution notes** documenting the decision. 6. Requires `ADMIN` role.

FR-10: Department Management (Master Data)

Field	Detail
ID	FR-10
Priority	High
Actor	Admin
Endpoints	`GET/POST/PUT/DELETE /api/admin/departments`
Description	Admin manages the master list of municipal departments.

Acceptance Criteria: 1. Admin can **Create** a new department (`name`, `description`). 2. Admin can **Read** a list of all departments and a single department by ID. 3. Admin can **Update** department details. 4. Admin can **Delete** a department (with guards if active tickets reference it). 5. Requires `ADMIN` role.

FR-11: User / Employee Management (Master Data)

Field	Detail
ID	FR-11
Priority	High
Actor	Admin
Endpoints	`GET/POST/PUT/DELETE /api/admin/users`
Description	Admin manages user accounts — primarily creating Worker and Officer accounts and mapping Workers to Departments.

Acceptance Criteria: 1. Admin can **Create** worker accounts with `role = DEPT_WORKER` and a `department_id`. 2. Admin can **Create** officer accounts with `role = OFFICER`. 3. Admin can **List** all users, filter by role or department. 4. Admin can **Update** user details (name, department mapping, active status). 5. Admin can **Deactivate** user accounts (soft delete). 6. Requires `ADMIN` role.

FR-12: Email Notifications

Field	Detail
ID	FR-12
Priority	Medium
Actor	System (Automated)
Description	System sends email notifications on every ticket status change.

Notification Triggers:

Status Transition	Recipient	Email Subject Template
`PENDING_AI` → `PENDING_ADMIN`	—	*(No notification — internal transition)*
`PENDING_ADMIN` → `ASSIGNED`	Citizen	"Your ticket #{{id}} has been assigned"
`ASSIGNED` → `RESOLVED`	Citizen	"Your ticket #{{id}} has been resolved"
`ASSIGNED` → `ESCALATED`	Admin	"Ticket #{{id}} escalated — review required"
`ESCALATED` → `REJECTED`	Citizen	"Your ticket #{{id}} has been closed"
`ESCALATED` → `ASSIGNED`	Worker	"Ticket #{{id}} has been re-assigned to you"

Acceptance Criteria: 1. Emails are sent via **Nodemailer** over SMTP. 2. Email sending can be **mocked** (logged to console) for the prototype. 3. The full notification logic pipeline must exist regardless of mocking.

FR-13: Officer Analytics Dashboard

Field	Detail
ID	FR-13
Priority	Medium
Actor	Officer
Endpoints	Multiple `GET /api/analytics/*` endpoints
Description	Read-only aggregated data endpoints powering the Officer's executive dashboard.

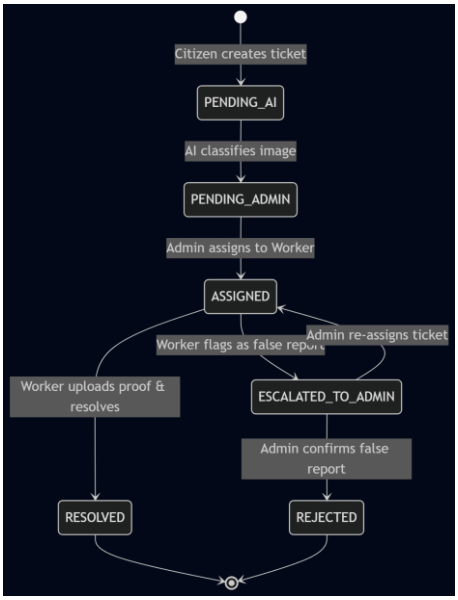
Required API Endpoints:

Endpoint	Description
`GET /api/analytics/tickets-by-department`	Count of tickets grouped by department.
`GET /api/analytics/tickets-by-status`	Count of tickets grouped by current status.
`GET /api/analytics/average-resolution-time`	Average time from `ASSIGNED` to `RESOLVED` per dept.
`GET /api/analytics/ai-accuracy-rate`	Percentage of times Admin accepted AI recommendation.
`GET /api/analytics/sla-compliance`	Percentage of tickets resolved within SLA thresholds.
`GET /api/analytics/monthly-trend`	Ticket count trend over the past 12 months.

Acceptance Criteria: 1. All endpoints return JSON-formatted aggregated data. 2. Frontend renders data as **charts** (bar, pie, line) and **KPI cards**. 3. Data is **read-only** — Officer cannot modify any records. 4. Requires `OFFICER` role (also accessible by `ADMIN`).

4.2 Ticket State Machine

The following state machine governs the complete lifecycle of a ticket:



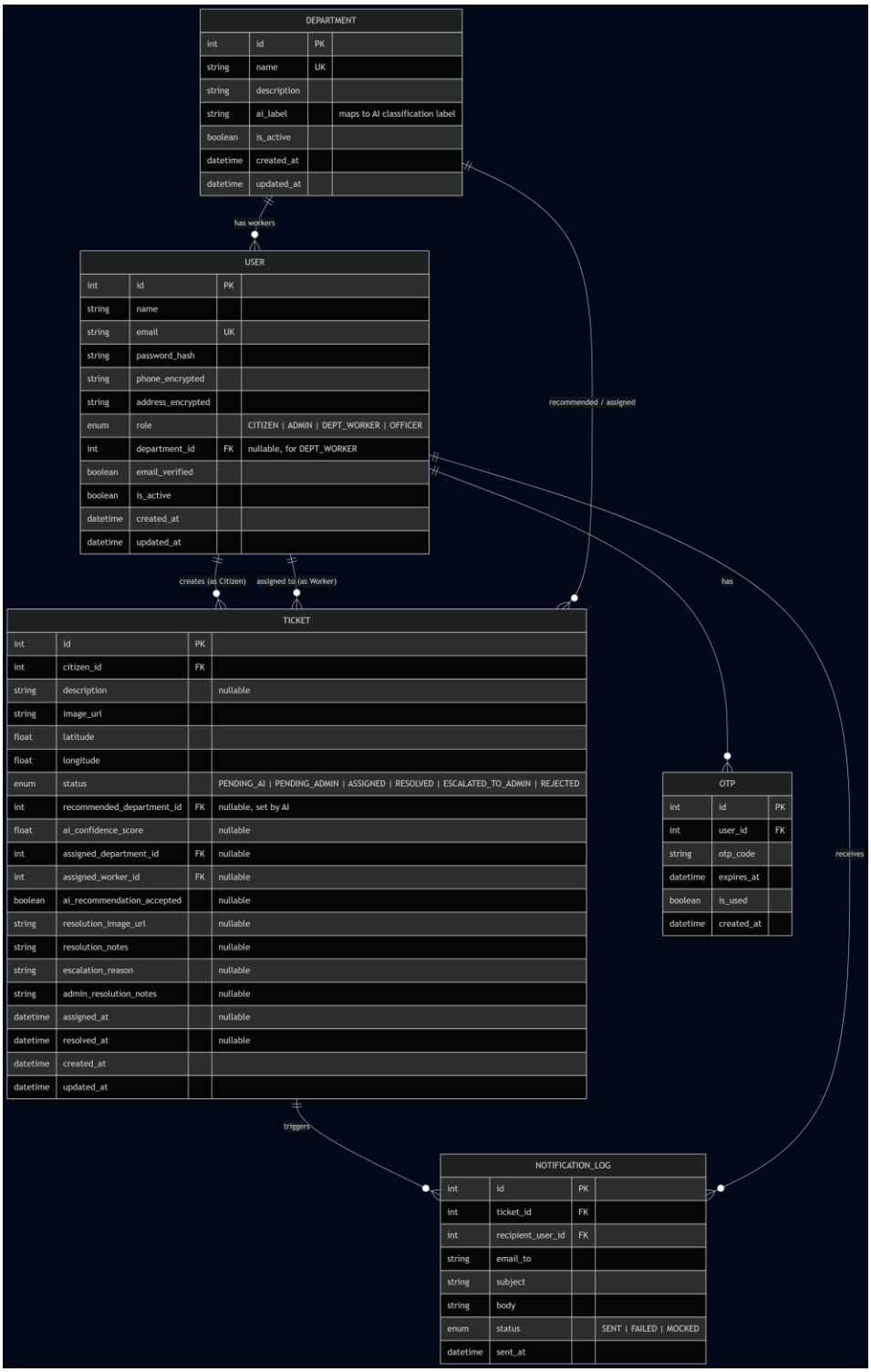
Valid State Transitions:

From	To	Triggered By	Condition
`PENDING_AI`	`PENDING_ADMIN`	System	AI classification completes
`PENDING_ADMIN`	`ASSIGNED`	Admin	Admin assigns to worker
`ASSIGNED`	`RESOLVED`	Worker	Proof uploaded + resolve action
`ASSIGNED`	`ESCALATED_TO_ADMIN`	Worker	False report flagged with reasoning
`ESCALATED_TO_ADMIN`	`REJECTED`	Admin	Confirms false / invalid report
`ESCALATED_TO_ADMIN`	`ASSIGNED`	Admin	Re-assigns to new worker/dept

---

5. Data Requirements

5.1 Entity-Relationship Diagram



5.2 Data Dictionary

Users Table

Column	Type	Constraints	Notes
`id`	INTEGER	PK, Auto-Increment	
`name`	VARCHAR(255)	NOT NULL	
`email`	VARCHAR(255)	NOT NULL, UNIQUE	
`password_hash`	VARCHAR(255)	NOT NULL	bcrypt hashed
`phone_encrypted`	TEXT	NULL	AES-256 encrypted
`address_encrypted`	TEXT	NULL	AES-256 encrypted
`role`	ENUM	NOT NULL	`CITIZEN`, `ADMIN`, `DEPT_WORKER`, `OFFICER`
`department_id`	INTEGER	FK → Departments, NULL	Required for `DEPT_WORKER` only
`email_verified`	BOOLEAN	DEFAULT false	
`is_active`	BOOLEAN	DEFAULT true	Soft-delete flag
`created_at`	TIMESTAMP	DEFAULT NOW()	
`updated_at`	TIMESTAMP	DEFAULT NOW()	

Departments Table

Column	Type	Constraints	Notes
`id`	INTEGER	PK, Auto-Increment	
`name`	VARCHAR(255)	NOT NULL, UNIQUE	e.g., “Water Supply”, “Roads”
`description`	TEXT	NULL	
`ai_label`	VARCHAR(100)	UNIQUE	Maps to AI classification output label
`is_active`	BOOLEAN	DEFAULT true	
`created_at`	TIMESTAMP	DEFAULT NOW()	

Column	Type	Constraints	Notes
updated_at	TIMESTAMP	DEFAULT NOW()	

Tickets Table

Column	Type	Constraints	Notes
id	INTEGER	PK, Auto-Increment	
citizen_id	INTEGER	FK → Users, NOT NULL	Creator of the ticket
description	TEXT	NULL	Optional citizen description
image_url	VARCHAR(500)	NOT NULL	Path/URL to uploaded image
latitude	DECIMAL(10,7)	NOT NULL	GPS latitude
longitude	DECIMAL(10,7)	NOT NULL	GPS longitude
status	ENUM	NOT NULL	See state machine
recommended_department_id	INTEGER	FK → Departments, NULL	Set by AI
ai_confidence_score	DECIMAL(5,4)	NULL	0.0000 to 1.0000
assigned_department_id	INTEGER	FK → Departments, NULL	Set by Admin
assigned_worker_id	INTEGER	FK → Users, NULL	Set by Admin
ai_recommendation_accepted	BOOLEAN	NULL	true if Admin accepted AI suggestion
resolution_image_url	VARCHAR(500)	NULL	Worker's "After" photo
resolution_notes	TEXT	NULL	Worker's notes
escalation_reason	TEXT	NULL	Worker's false-report reasoning
admin_resolution_notes	TEXT	NULL	Admin's escalation decision notes
assigned_at	TIMESTAMP	NULL	When ticket was assigned
resolved_at	TIMESTAMP	NULL	When ticket was resolved/rejected
created_at	TIMESTAMP	DEFAULT NOW()	

Column	Type	Constraints	Notes
`updated_at`	TIMESTAMP	DEFAULT NOW()	

Notification Log Table

Column	Type	Constraints	Notes
`id`	INTEGER	PK, Auto-Increment	
`ticket_id`	INTEGER	FK → Tickets	
`recipient_user_id`	INTEGER	FK → Users	
`email_to`	VARCHAR(255)	NOT NULL	
`subject`	VARCHAR(500)	NOT NULL	
`body`	TEXT	NOT NULL	
`status`	ENUM	NOT NULL	`SENT`, `FAILED`, `MOCKED`
`sent_at`	TIMESTAMP	DEFAULT NOW()	

OTP Table

Column	Type	Constraints	Notes
`id`	INTEGER	PK, Auto-Increment	
`user_id`	INTEGER	FK → Users	
`otp_code`	VARCHAR(6)	NOT NULL	
`expires_at`	TIMESTAMP	NOT NULL	
`is_used`	BOOLEAN	DEFAULT false	
`created_at`	TIMESTAMP	DEFAULT NOW()	

---

6. External Interface Requirements

6.1 User Interface Requirements

UI-01: Authentication Pages

- Login form: email + password fields, “Login” button, link to registration.
- Registration form: name, email, password, phone, address fields, “Register” button.
- OTP verification form: 6-digit code input, “Verify” button, “Resend OTP” link.

UI-02: Citizen Dashboard

- **Create Ticket**: Image upload drag-and-drop area, GPS auto-capture indicator, optional description textarea, "Submit" button.
- **My Tickets**: Table/card view listing citizen's own tickets with columns: ID, Status (color-coded badge), Department, Created Date, Last Updated.
- **Ticket Detail**: Full ticket info, image preview, map pin for GPS location, status timeline.

UI-03: Admin Dashboard

- **Pending Review Queue**: List of `PENDING_ADMIN` tickets with AI recommendation, confidence score, image preview.
- **Assignment Modal**: Department dropdown (pre-filled with AI recommendation), Worker dropdown (filtered by department), "Assign" button.
- **Escalation Queue**: List of `ESCALATED_TO_ADMIN` tickets with worker's reasoning, "Reject" and "Re-assign" actions.
- **Master Data Management**: Departments CRUD table, Users CRUD table with role and department filters.

UI-04: Worker Dashboard

- **My Tasks**: List of tickets assigned to the worker, filtered by `ASSIGNED` status.
- **Resolve Modal**: Image upload for "After" photo, resolution notes textarea, "Mark Resolved" button.
- **Flag Modal**: Reasoning textarea (required), "Flag as False Report" button.

UI-05: Officer Analytics Dashboard

- **KPI Cards**: Total tickets, resolved tickets, average resolution time, AI accuracy rate.
- **Charts**: Tickets by department (bar chart), tickets by status (pie chart), monthly trend (line chart), SLA compliance (gauge or bar).
- **Filters**: Date range selector, department filter.

6.2 API Interface Requirements

All APIs follow **RESTful** conventions: - Base URL: `/api` - Content-Type: `application/json` (except file uploads: `multipart/form-data`) - Authentication: `Authorization: Bearer <JWT>` header on all protected routes. - Standard error response format:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Human-readable error message",
    "details": []
  }
}
```

Complete API Endpoint Summary:

Method	Endpoint	Role(s)	Description
POST	`/api/auth/register`	Public / Admin	Register new user
POST	`/api/auth/login`	Public	Login and receive JWT
POST	`/api/auth/send-otp`	Authenticated	Request email OTP
POST	`/api/auth/verify-otp`	Authenticated	Verify email OTP
POST	`/api/tickets`	Citizen	Create new ticket
GET	`/api/tickets`	All (filtered)	List tickets (role-based filtering)
GET	`/api/tickets/:id`	All (authorized)	Get ticket detail
PUT	`/api/tickets/:id/assign`	Admin	Assign ticket to worker
PUT	`/api/tickets/:id/resolve`	Dept Worker	Resolve ticket with proof
PUT	`/api/tickets/:id/flag-false`	Dept Worker	Flag ticket as false report
PUT	`/api/tickets/:id/resolve-escalation`	Admin	Handle escalated ticket
GET	`/api/admin/departments`	Admin	List departments
POST	`/api/admin/departments`	Admin	Create department
PUT	`/api/admin/departments/:id`	Admin	Update department
DELETE	`/api/admin/departments/:id`	Admin	Delete department
GET	`/api/admin/users`	Admin	List users
POST	`/api/admin/users`	Admin	Create user account
PUT	`/api/admin/users/:id`	Admin	Update user
DELETE	`/api/admin/users/:id`	Admin	Deactivate user
GET	`/api/analytics/tickets-by-department`	Officer, Admin	Tickets grouped by department
GET	`/api/analytics/tickets-by-status`	Officer, Admin	Tickets grouped by status
GET	`/api/analytics/average-resolution-time`	Officer, Admin	Avg resolution time per department
GET	`/api/analytics/ai-accuracy-rate`	Officer, Admin	AI recommendation acceptance rate
GET	`/api/analytics/sla-compliance`	Officer, Admin	SLA compliance metrics

Method	Endpoint	Role(s)	Description
GET	`/api/analytics/monthly-trend`	Officer, Admin	Monthly ticket volume trend

6.3 Hardware Interface Requirements

- **GPS**: Browser Geolocation API (`navigator.geolocation`) for capturing citizen's location during ticket creation.
- **Camera/File System**: Browser File API for image upload (camera capture on mobile, file picker on desktop).

6.4 Software Interface Requirements

External Service	Interface Type	Purpose
Google Cloud Vision API	REST API	Image classification (primary option)
TensorFlow.js pre-trained model	JS Library	Image classification (alternative option)
SMTP Server (via Nodemailer)	SMTP Protocol	Sending email notifications
PostgreSQL Database	TCP/ORM	Persistent data storage

---

7. Non-Functional Requirements

7.1 Performance Requirements

ID	Requirement
NFR-01	API response time ≤ 500ms for standard CRUD operations (p95).
NFR-02	AI image classification should complete within ≤ 5 seconds.
NFR-03	The system should support at least **100 concurrent users** for the prototype.
NFR-04	Image upload size capped at **10MB** per file.

7.2 Security Requirements

ID	Requirement
NFR-05	Passwords must be hashed with **bcrypt** (cost factor ≥ 10).
NFR-06	Sensitive PII fields (phone, address) must be encrypted at rest using **AES-256**.
NFR-07	All API routes (except `/auth/register` and `/auth/login`) require a valid **JWT**.
NFR-08	RBAC middleware must enforce role-based access on every protected endpoint.
NFR-09	JWT tokens must have a **configurable expiration** and include role information.
NFR-10	Input validation and sanitization on all user-provided data to prevent SQL

ID	Requirement
	injection and XSS.
NFR-11	File uploads must validate **MIME type** (JPEG/PNG only) and **file size**.
NFR-12	CORS must be configured to allow only the frontend origin.

7.3 Reliability & Availability

ID	Requirement
NFR-13	The system should handle **graceful failures** — if AI service is down, tickets still proceed to `PENDING_ADMIN` with manual review flag.
NFR-14	Database transactions must be used for multi-step operations (e.g., ticket assignment + notification).
NFR-15	All API errors must return structured error responses (no stack traces in production).

7.4 Usability

ID	Requirement
NFR-16	Frontend must be **responsive** (desktop + tablet + mobile).
NFR-17	Ticket statuses should be **color-coded** for quick visual identification.
NFR-18	Form validation with inline error messages on all user input forms.

7.5 Scalability

ID	Requirement
NFR-19	Database schema must be designed for easy extension (new departments, new statuses).
NFR-20	AI service integration must be abstracted behind a service layer for easy swapping.

7.6 Maintainability

ID	Requirement
NFR-21	Codebase must follow a **modular architecture** (routes, controllers, services, models).
NFR-22	Code must be **well-commented** and ready for production-like review.
NFR-23	Environment variables managed via `.env` file (never hardcoded secrets).